Memoria de Práctica: Laboratorio 1 - Ingesta de Fuentes Heterogéneas

Módulo: Sistemas de Big Data (IABD)

Alumno: Ángel Postigo Rodríguez

1. Parte A: Ingesta BATCH con Sqoop

El objetivo de esta sección es migrar la tabla histórica de pedidos desde una base de datos relacional (MySQL) hacia el sistema de archivos distribuido (HDFS) utilizando Apache Sqoop.

Comando de Ingesta (Sqoop)

```
sqoop import \
--connect jdbc:mysql://mysql/ventas_db \
--username sqoopuser \
--password 'sqooppassword' \
--table pedidos \
--target-dir /user/iabd_admin/ingesta/ventas/pedidos \
--m 1
```

Evidencias de Ejecución (Sqoop)

```
bash-4.2$ hdfs dfs -ls /user/iabd_admin/ingesta/ventas/pedidos
Found 2 items
-rw-r--r--   1 hadoop supergroup          0 2026-11-04 08:26 /user/iabd_admin/ingesta/ventas/pedidos/_SUCCESS
-rw-r--r--   1 hadoop supergroup      24803 2026-11-04 08:26 /user/iabd_admin/ingesta/ventas/pedidos/part-m-00000
bash-4.2$
```

```
PS C:\Users\angel\master-ia-bigdata-master> docker exec -it namenode bash
bash-4.2$ hdfs dfs -cat /user/iabd_admin/ingesta/ventas/pedidos/part-m-00000
1,Producto_1,5,49.75
2,Producto_2,2,44.59
3,Producto_3,7,99.41
4,Producto_4,10,87.44
5,Producto_5,5,66.74
6,Producto_6,10,81.03
7,Producto_7,2,38.88
8,Producto_8,5,9.42
9,Producto_9,2,29.07
10,Producto_10,2,67.52
11,Producto_11,1,21.26
12,Producto_12,10,96.25
13,Producto_13,1,37.37
14,Producto_14,8,43.81
15,Producto_15,1,95.99
16,Producto_16,7,29.48
17,Producto_17,6,96.31
18,Producto_18,2,64.50
19,Producto_19,10,58.73
20,Producto_20,3,35.43
21,Producto_21,2,44.61
22,Producto_22,10,28.70
23,Producto_23,7,43.78
24,Producto_24,3,73.59
25,Producto_25,1,4.14
26,Producto_26,1,16.38
27,Producto_27,7,73.95
28,Producto_28,8,57.67
29,Producto_29,7,30.11
30,Producto_30,7,53.08
31,Producto_31,6,37.57
32,Producto_32,1,36.16
33,Producto_33,6,48.90
34,Producto_34,10,3.57
35,Producto_35,5,27.71
36,Producto_36,10,94.32
37,Producto_37,9,42.68
38,Producto_38,6,62.13
39,Producto_39,4,96.28
40,Producto_40,8,70.21
41,Producto_41,4,67.53
42,Producto_42,4,51.06
43,Producto_43,7,62.46
44,Producto_44,3,27.23
45,Producto_45,7,55.18
46,Producto_46,8,3.40
47,Producto_47,10,67.91
48,Producto_48,6,59.63
49,Producto_49,5,21.85
50,Producto_50,9,78.43
51,Producto_51,3,93.58
52,Producto_52,10,72.17
53,Producto_53,9,29.03
```

2. Parte B: Ingesta STREAMING con Flume

En esta sección, se implementa un agente de Apache Flume para capturar de forma continua (streaming) los registros simulados de un servidor web, moviéndolos atómicamente desde un directorio de spooling local hacia HDFS.

Archivo de Configuración (flume-conf.properties)

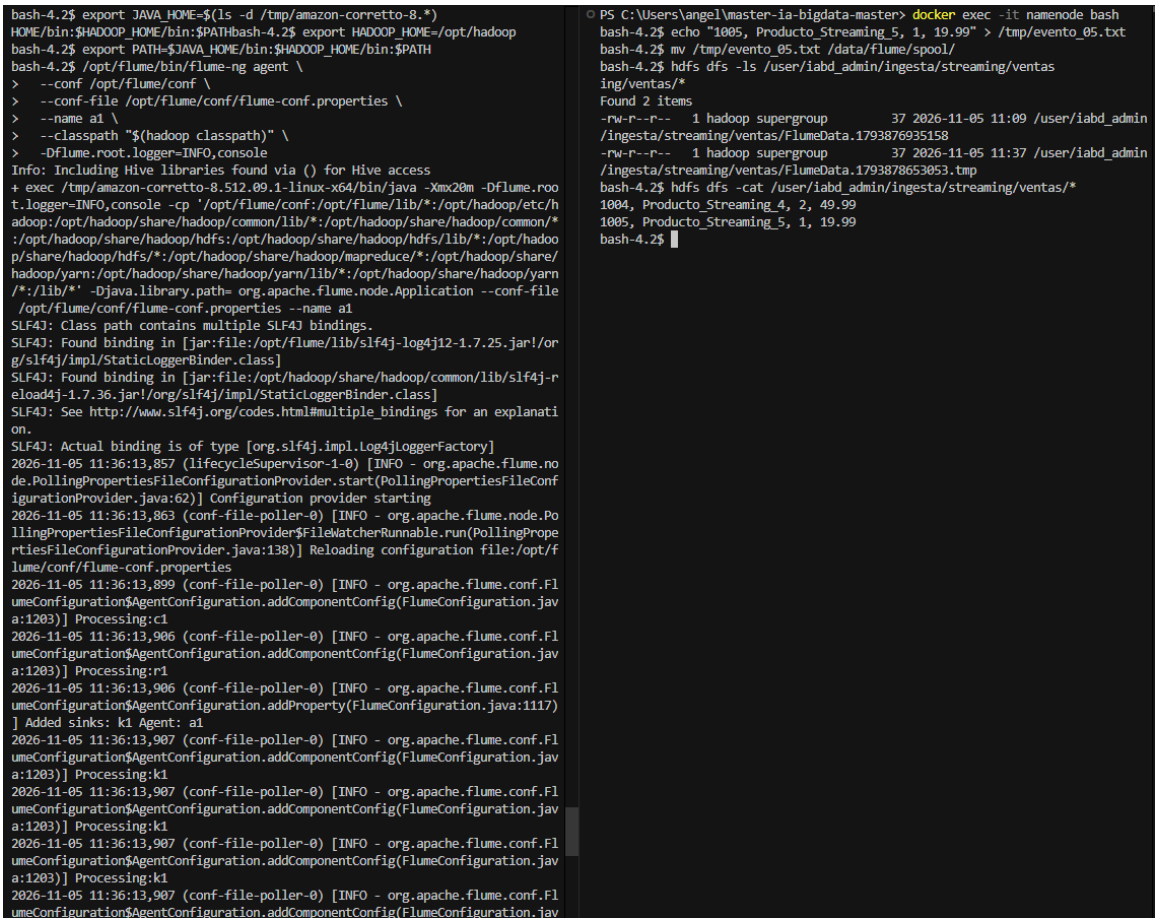
```
a1.sources = r1
a1.channels = c1
a1.sinks = k1

a1.sources.r1.type = spooldir
a1.sources.r1.spoolDir = /data/flume/spool
a1.sources.r1.fileHeader = true
a1.sources.r1.channels = c1

a1.channels.c1.type = memory
a1.channels.c1.capacity = 1000
a1.channels.c1.transactionCapacity = 100

a1.sinks.k1.type = hdfs
a1.sinks.k1.hdfs.path                                                    =
hdfs://namenode:8020/user/iabd_admin/ingesta/streaming/ventas
a1.sinks.k1.hdfs.fileType = DataStream
a1.sinks.k1.hdfs.writeFormat = Text
a1.sinks.k1.hdfs.rollInterval = 60
a1.sinks.k1.hdfs.rollSize = 0
a1.sinks.k1.hdfs.rollCount = 0
a1.sinks.k1.hdfs.useLocalTimeStamp = true
a1.sinks.k1.channel = c1
```

Evidencias de Ejecución (Flume)



3. Discusión y Cierre

¿Por qué no usamos Sqoop para los logs? ¿Qué pasaría?

Apache Sqoop está diseñado exclusivamente para la transferencia masiva de datos en bloque (Batch) desde sistemas relacionales estructurados (RDBMS compatibles con JDBC) hacia HDFS. Los logs son datos semiestructurados generados en tiempo real y de forma continua (streaming). Si utilizáramos Sqoop, deberíamos ejecutarlo periódicamente en micro-lotes. Cada ejecución levantaría un trabajo MapReduce completo (arranque de JVM, asignación de contenedores YARN, planificación de mappers), lo que introduciría una latencia inasumible, un consumo masivo e ineficiente de recursos en el clúster y un alto riesgo de pérdida de datos no comprometidos en disco.

¿Por qué no usamos Flume para la base de datos? ¿Se podría?

Técnicamente existen conectores o plugins comunitarios (como flume-ng-sql-source) para realizar sondeos mediante sentencias SELECT periódicas. Sin embargo, no es la herramienta arquitectónicamente adecuada. Flume no está concebido para gestionar la consistencia relacional, la integridad transaccional ni el particionado paralelo por claves primarias que sí domina Sqoop. Además, realizar consultas continuas de extracción sobre tablas relacionales masivas sobrecargaría innecesariamente el motor transaccional (OLTP) de la base de datos origen.

En Flume, usamos un memory channel. ¿Qué riesgo tiene esto en un entorno de producción real?

El riesgo crítico es la falta absoluta de durabilidad ante fallos (data loss). La memoria RAM es un medio de almacenamiento volátil. Si el proceso del agente de Flume sufre una caída imprevista (crash), se agota la memoria del host (Out Of Memory) o el contenedor se reinicia abruptamente mientras hay eventos retenidos en el canal, todos los registros encolados que aún no hayan sido confirmados y persistidos por el sumidero (Sink) en HDFS se pierden de forma irremediable. Para solventarlo en un entorno productivo, se debe emplear un "file channel" que persiste los eventos temporalmente en el disco local antes de enviarlos, garantizando transaccionalidad y tolerancia a caídas, o delegar el transporte en un sistema distribuido tolerante a fallos como Apache Kafka.